

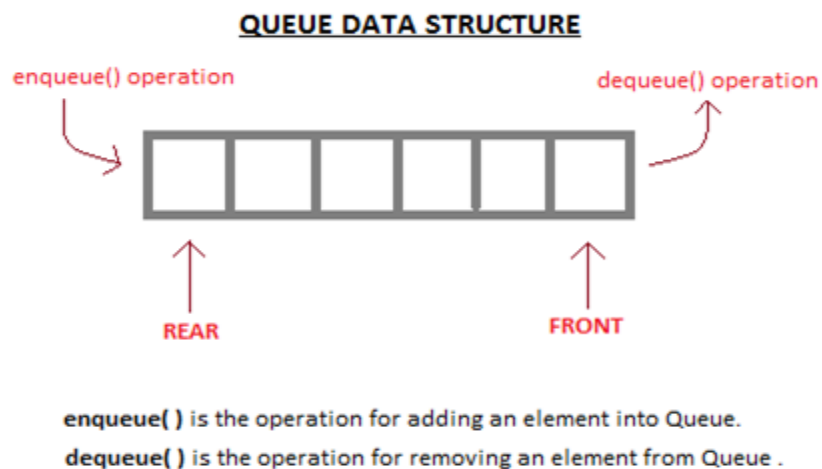
UNIT IV

QUEUE

Queue is also an abstract data type or a linear data structure, just like stack data structure, in which the first element is inserted from one end called the **REAR**(also called **tail**), and the removal of existing element takes place from the other end called as **FRONT**(also called **head**).

This makes queue as **FIFO** (First in First Out) data structure, which means that element inserted first will be removed first.

Which is exactly how queue system works in real world? If you go to a ticket counter to buy movie tickets, and are first in the queue, then you will be the first one to get the tickets. The process to add an element into queue is called **Enqueue** and the process of removal of an element from queue is called **Dequeue**.

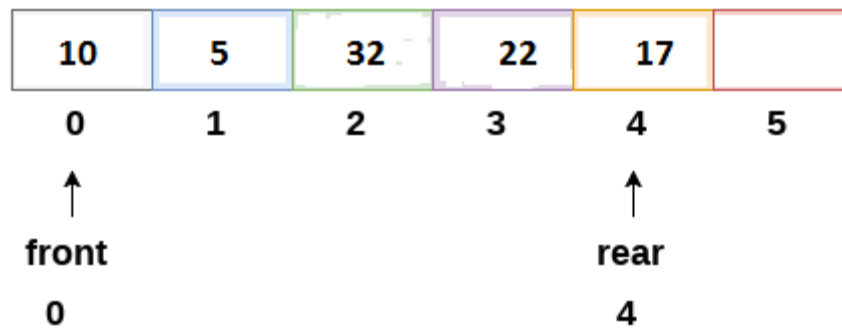


REPRESENTATION OF QUEUES

Queues can be implemented by using either arrays or linked lists. In this section, we will see how queues are implemented using each of these data structures.

i. Array Representation of Queues

We can easily represent queue by using linear arrays. There are two variables i.e. front and rear, that are implemented in the case of every queue. Front and rear variables point to the position from where insertions and deletions are performed in a queue. Initially, the value of front and queue is -1 which represents an empty queue. For example, an array representation of a queue containing 5 elements along with the respective values of front and rear, is shown in the following figure.

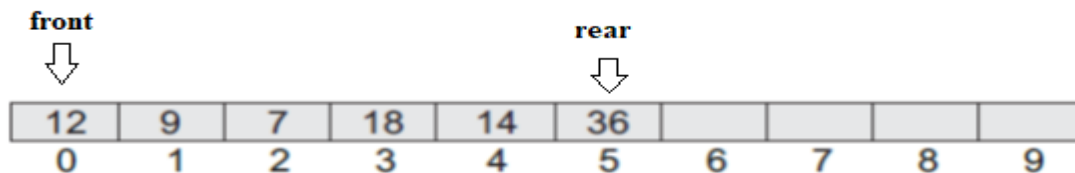


Operations on Queues

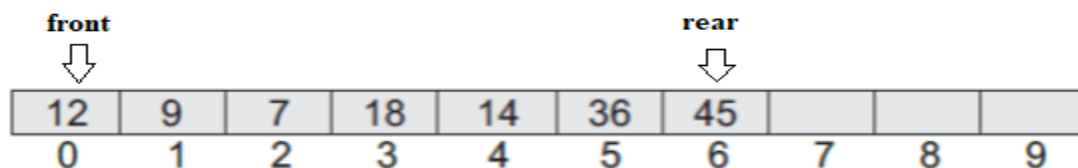
Initially the **head** (FRONT) and the **tail** (REAR) of the queue points at the first index of the array (starting the index of array from 0). As we add elements to the queue, the REAR keeps on moving ahead, always pointing to the position where the next element will be inserted, while the FRONT remains at the first index.

a. INSERTION(ENQUEUE)

The array representation of a queue is shown in Fig.



In this queue FRONT = 0 and REAR = 5. Suppose we want to add another element with value 45, then REAR would be incremented by 1 and the value would be stored at the position pointed by REAR. The queue after addition would be as shown in Fig.



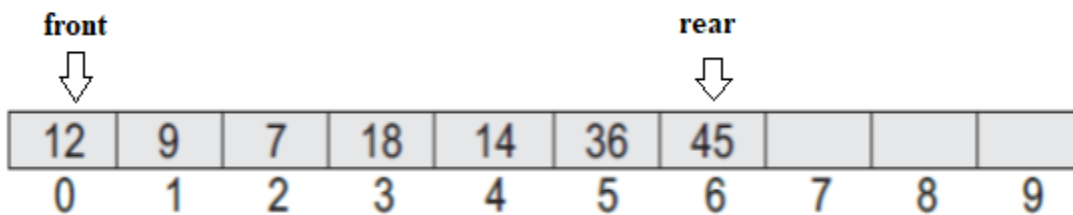
Here, FRONT = 0 and REAR = 6. Every time a new element has to be added, we repeat the same procedure. However, before inserting an element in a queue, we must check for overflow conditions. An overflow will occur when we try to insert an element into a queue that is already full. When $REAR = MAX - 1$, where MAX is the size of the queue, we have an overflow condition. Note that we have written $MAX - 1$ because the index starts from 0.

Algorithm to insert an element in a queue

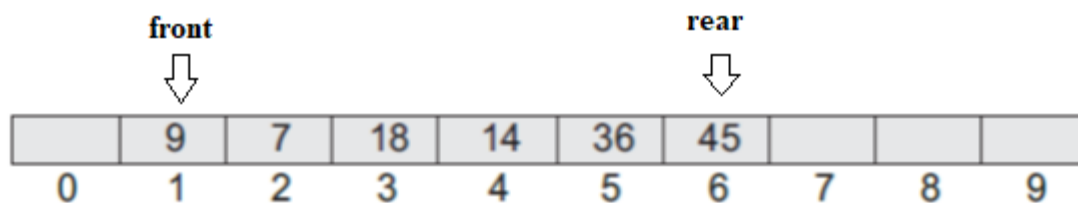
```
Step 1: IF REAR = MAX-1
        Write OVERFLOW
        Goto step 4
    [END OF IF]
Step 2: IF FRONT = -1 and REAR = -1
        SET FRONT = REAR = 0
    ELSE
        SET REAR = REAR + 1
    [END OF IF]
Step 3: SET QUEUE[REAR] = NUM
Step 4: EXIT
```

b. DELETE(DEQUEUE)

Initially the queue contain the data and its front and rear positions are



If we want to delete an element from the queue, then the value of FRONT will be incremented. Deletions are done from only this end of the queue. The queue after deletion will be as shown in Fig.



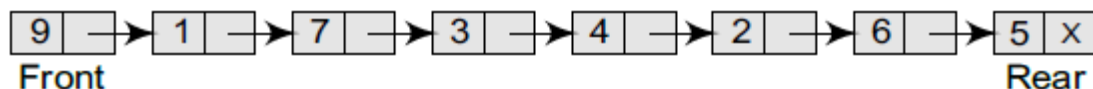
Before deleting an element from a queue, we must check for underflow conditions. An underflow condition occurs when we try to delete an element from a queue that is already empty. If FRONT = -1 and REAR = -1, it means there is no element in the queue.

Algorithm to delete an element from a queue

```
Step 1: IF FRONT = -1 OR FRONT > REAR
        Write UNDERFLOW
    ELSE
        SET VAL = QUEUE[FRONT]
        SET FRONT = FRONT + 1
    [END OF IF]
Step 2: EXIT
```

ii. Linked Representation of Queues

In a linked queue, every element has two parts, one that stores the data and another that stores the address of the next element. The START pointer of the linked list is used as FRONT. Here, we will also use another pointer called REAR, which will store the address of the last element in the queue. All insertions will be done at the rear end and all the deletions will be done at the front end. If FRONT = REAR = NULL, then it indicates that the queue is empty. The linked representation of a queue is shown in Fig.

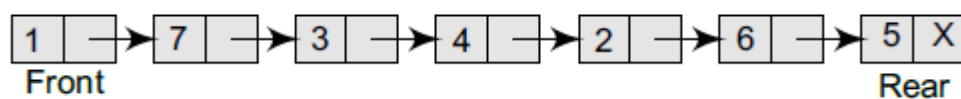


Operations on Linked Queues

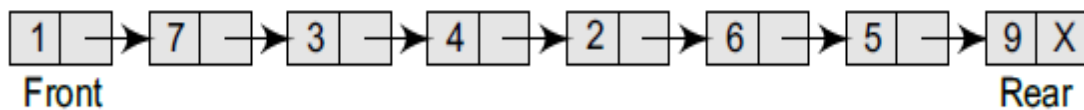
A queue has two basic operations: insert and delete. The insert operation adds an element to the end of the queue, and the delete operation removes an element from the front or the start of the queue. Apart from this, there is another operation peek which returns the value of the first element of the queue.

a. Insert Operation(Enqueue)

The insert operation is used to insert an element into a queue. The new element is added as the last element of the queue. Consider the linked queue shown in Fig.



To insert an element with value 9, we first check if $FRONT = NULL$. If the condition holds, then the queue is empty. So, we allocate memory for a new node; store the value in its data part and $NULL$ in its next part. The new node will then be called both $FRONT$ and $rear$. However, if $FRONT \neq NULL$, then we will insert the new node at the rear end of the linked queue and name this new node as $rear$. Thus, the updated queue becomes as shown in Fig.



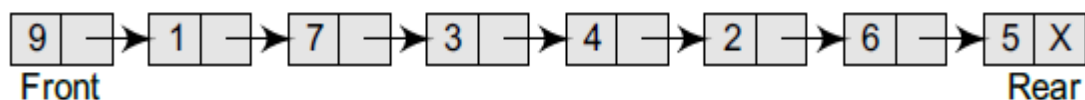
Algorithm to insert an element in a linked queue

```

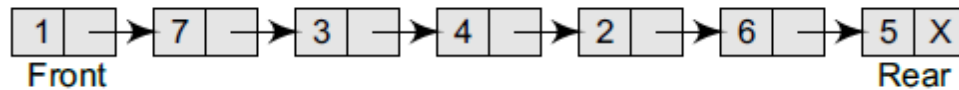
Step 1: Allocate memory for the new node and name
        it as PTR
Step 2: SET PTR → DATA = VAL
Step 3: IF FRONT = NULL
        SET FRONT = REAR = PTR
        SET FRONT → NEXT = REAR → NEXT = NULL
      ELSE
        SET REAR → NEXT = PTR
        SET REAR = PTR
        SET REAR → NEXT = NULL
      [END OF IF]
Step 4: END
  
```

b. Delete Operation(Dequeue)

The delete operation is used to delete the element that is first inserted in a queue, i.e., the element whose address is stored in $FRONT$. However, before deleting the value, we must first check if $FRONT = NULL$ because if this is the case, then the queue is empty and no more deletions can be done. If an attempt is made to delete a value from a queue that is already empty, an underflow message is printed. Consider the queue shown in Fig.



To delete an element, we first check if $FRONT = NULL$. If the condition is false, then we delete the first node pointed by $FRONT$. The $FRONT$ will now point to the second element of the linked queue. Thus, the updated queue becomes as shown in Fig.



Algorithm to delete an element from a linked queue

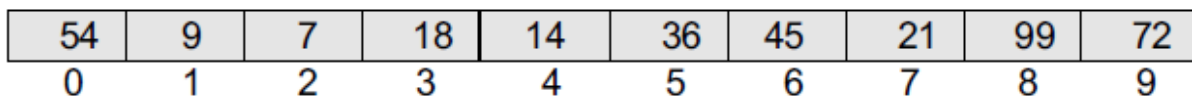
```

Step 1: IF FRONT = NULL
        Write "Underflow"
        Go to Step 5
      [END OF IF]
Step 2: SET PTR = FRONT
Step 3: SET FRONT = FRONT -> NEXT
Step 4: FREE PTR
Step 5: END
  
```

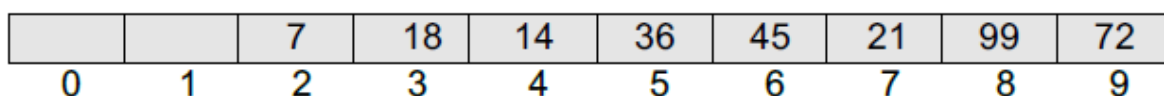
1. TYPES OF QUEUES

i. CIRCULAR QUEUES

In linear queues, insertions can be done only at one end called the $REAR$ and deletions are always done from the other end called the $FRONT$. Look at the queue shown in Fig. Here, $FRONT = 0$ and $REAR = 9$.

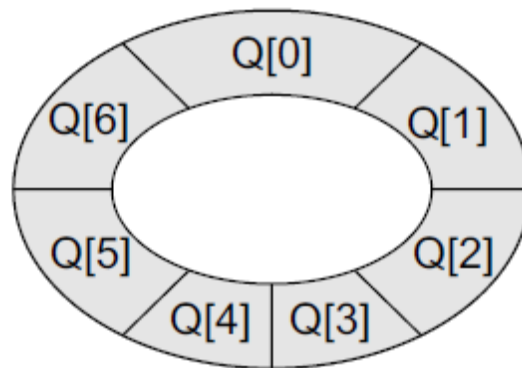


Now, if you want to insert another value, it will not be possible because the queue is completely full. There is no empty space where the value can be inserted. Consider a scenario in which two successive deletions are made. The queue will then be given as shown in Fig. Here, $front = 2$ and $REAR = 9$.



Suppose we want to insert a new element in the queue shown in Fig. Even though there is space available, the overflow condition still exists because the condition $\text{rear} = \text{MAX} - 1$ still holds true. This is a major drawback of a linear queue.

To resolve this problem, we have two solutions. First, shift the elements to the left so that the vacant space can be occupied and utilized efficiently. But this can be very time-consuming, especially when the queue is quite large. The second option is to use a circular queue. In the circular queue, the first index comes right after the last index. Conceptually, you can think of a circular queue as shown in Fig.



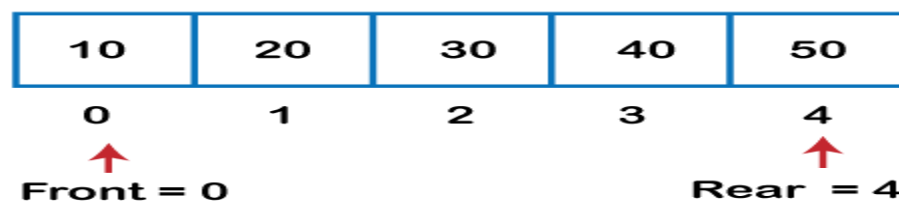
The circular queue will be full only when $\text{front} = 0$ and $\text{rear} = \text{Max} - 1$. A circular queue is implemented in the same manner as a linear queue is implemented. The only difference will be in the code that performs insertion and deletion operations.

Example:-

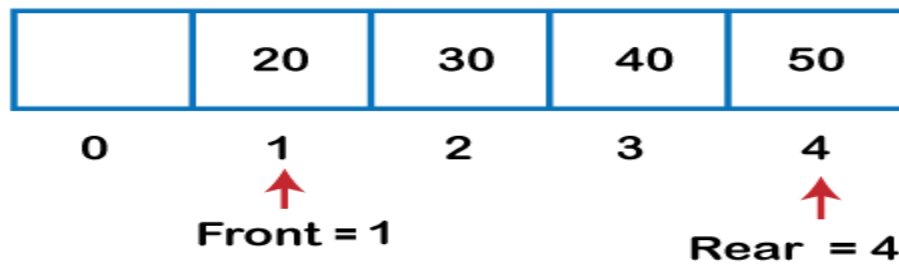
Initially the queue is empty and the size of the queue is 5. In this position front and rear are -1.



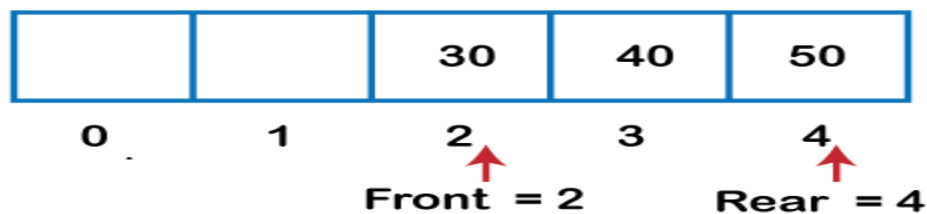
After inserting the values 10, 20, 30, 40, 50 the front position is at 0 and rear position is at 4. Now the queue is full.



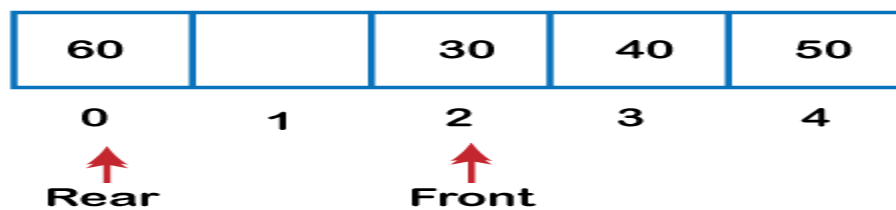
Now apply dequeue operation the front element 10 is deleted and the front position move to 1.



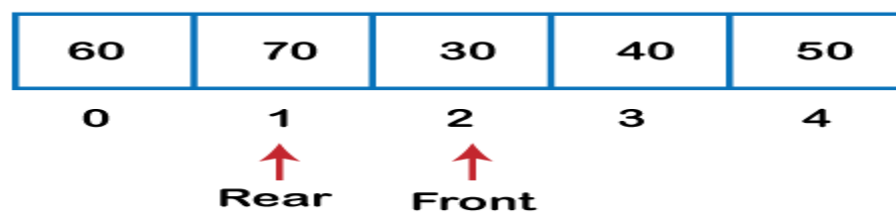
Again apply the dequeue operation 20 is deleted and the front position move to 2



Now insert 60 into the queue it is placed in to the first position and rear is also move to the first position of the queue.



Now insert data 70 in to the queue, It's placed to the second position of the queue and rear is also increased. Now the queue is full.



Enqueue operation (Insertion)

First, we will check whether the Queue is full or not. Initially the front and rear are set to -1. When we insert the first element in a Queue, front and rear both are set to 0. When we insert a new element, the rear gets incremented, i.e., **rear=rear+1**.

Scenarios for inserting an element

There are two scenarios in which queue are not full:

- If $\text{rear} \neq \text{max} - 1$, then rear will be incremented to $\text{mod}(\text{maxsize})$ and the new value will be inserted at the rear end of the queue.
- If $\text{front} = 0$ and $\text{rear} = \text{max} - 1$, it means that queue is not full, then set the value of rear to 0 and insert the new element there.

There are two cases in which the element cannot be inserted:

- When $\text{front} = 0$ & $\text{rear} = \text{max} - 1$, which means that front is at the first position of the Queue and rear is at the last position of the Queue.
- $\text{front} = \text{rear} + 1$;

Algorithm to insert an element in a circular queue

Step 1: IF $(\text{REAR} + 1) \% \text{MAX} = \text{FRONT}$

Write " OVERFLOW "

Goto step 4

[End OF IF]

Step 2: IF $\text{FRONT} = -1$ and $\text{REAR} = -1$

SET $\text{FRONT} = \text{REAR} = 0$

ELSE IF $\text{REAR} = \text{MAX} - 1$ and $\text{FRONT} \neq 0$

SET $\text{REAR} = 0$

ELSE

SET $\text{REAR} = (\text{REAR} + 1) \% \text{MAX}$

[END OF IF]

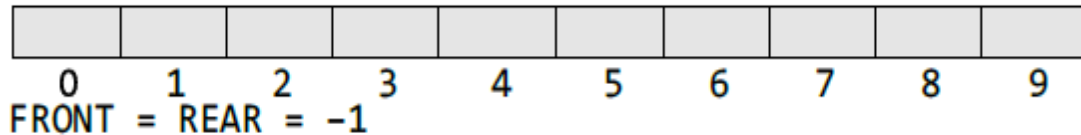
Step 3: SET $\text{QUEUE}[\text{REAR}] = \text{VAL}$

Step 4: EXIT

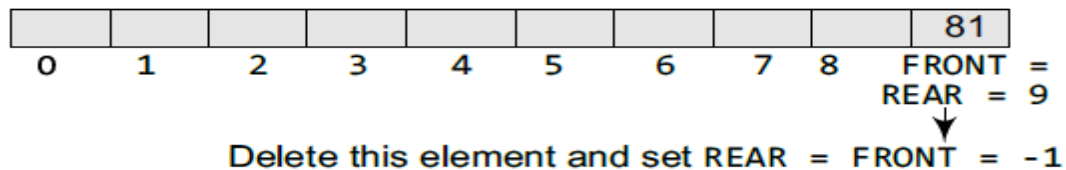
Dequeue Operation (Deletion)

To delete an element from circular linked list, we check for three conditions.

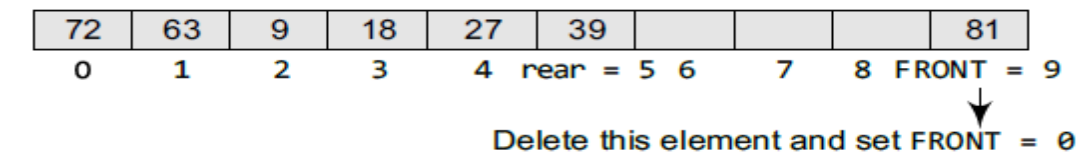
- i. If $\text{front} = -1$, then there are no elements in the queue. So, an underflow condition will be reported.



- ii. If the queue is not empty and $\text{front} = \text{rear}$, then after deleting the element at the front the queue becomes empty and so front and rear are set to -1 . This is illustrated in Fig.



- iii. If the queue is not empty and $\text{front} = \text{MAX}-1$, then after deleting the element at the front, front is set to 0. This is shown in Fig.



Algorithm to delete an element from the circular queue

Step 1: IF $\text{FRONT} = -1$

Write " UNDERFLOW "

Goto Step 4

[END of IF]

Step 2: SET $\text{VAL} = \text{QUEUE}[\text{FRONT}]$

Step 3: IF $\text{FRONT} = \text{REAR}$

SET $\text{FRONT} = \text{REAR} = -1$

ELSE

IF $\text{FRONT} = \text{MAX} - 1$

SET $\text{FRONT} = 0$

ELSE

SET $\text{FRONT} = \text{FRONT} + 1$

[END of IF]

[END OF IF]

Step 4: EXIT

ii. Priority Queues

A priority queue is a data structure in which each element is assigned a priority. The priority of the element will be used to determine the order in which the elements will be processed. The general rules of processing the elements of a priority queue are

- Every element in a priority queue has some priority associated with it.
- An element with the higher priority will be deleted before the deletion of the lesser priority.
- If two elements in a priority queue have the same priority, they will be arranged using the FIFO principle.

A priority queue can be thought of as a modified queue in which when an element has to be removed from the queue, the one with the highest-priority is retrieved first. The priority of the element can be set based on various factors. Priority queues are widely used in operating systems to execute the highest priority process first. The priority of the process may be set based on the CPU time it requires to get executed completely.

Implementation of a Priority Queue

There are two ways to implement a priority queue.

- i. We can use a sorted list to store the elements so that when an element has to be taken out, the queue will not have to be searched for the element with the highest priority.
- ii. We can use an unsorted list so that insertions are always done at the end of the list. Every time when an element has to be removed from the list, the element with the highest priority will be searched and removed.

Linked Representation of a Priority Queue

When a priority queue is implemented using a linked list, then every node of the list will have three parts:

- (a) The information or data part
- (b) The priority number of the element, and
- (c) The address of the next element.

If we are using a sorted linked list, then the element with the higher priority will precede the element with the lower priority. Consider the priority queue shown in Fig.



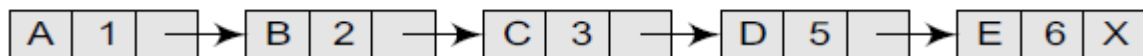
For example, if there are two elements A and B, where A has a priority number 1 and B has a priority number 5, then A will be processed before B as it has higher priority than B.

The priority queue in Fig. below is a sorted priority queue having six elements. From the queue, we cannot make out whether A was inserted before E or whether E joined the queue before A because the list is not sorted based on FCFS. Here, the element with a higher priority comes before the element with a lower priority. However, we can definitely say that C was inserted in the queue before D because when two elements have the same priority the elements are arranged and processed on FCFS principle.



Insertion

When a new element has to be inserted in a priority queue, we have to traverse the entire list until we find a node that has a priority lower than that of the new element. The new node is inserted before the node with the lower priority. However, if there exists an element that has the same priority as the new element, the new element is inserted after that element. For example, consider the priority queue shown in Fig.



If we have to insert a new element with data = F and priority number = 4, then the element will be inserted before D that has priority number 5, which is lower priority than that of the new element. So, the priority queue now becomes as shown in Fig.



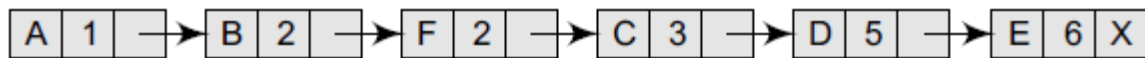
Priority queue after insertion of a new node

However, if we have a new element with data = F and priority number = 2, then the element will be inserted after B, as both these elements have the same priority but the insertions are done on FCFS basis as shown in Fig.



Deletion

Deletion is a very simple process in this case. The first node of the list will be deleted and the data of that node will be processed first. Consider a priority queue given below



After deletion the priority queue is



Array Representation of a Priority Queue

When arrays are used to implement a priority queue, then a separate queue for each priority number is maintained. Each of these queues will be implemented using circular arrays or circular queues. Every individual queue will have its own FRONT and REAR pointers.

We use a two-dimensional array for this purpose where each queue will be allocated the same amount of space. Look at the two-dimensional representation of a priority queue given below. Given the front and rear values of each queue, the two-dimensional matrix can be formed as shown in Fig.

FRONT	REAR		1	2	3	4	5
3	3	1			A		
1	3	2	B	C	D		
4	5	3				E	F
4	1	4	I			G	H

FRONT[K] and REAR[K] contain the front and rear values of row K, where K is the priority number. Note that here we are assuming that the row and column indices start from 1, not 0. Obviously, while programming, we will not take such assumptions.

Insertion

To insert a new element with priority K in the priority queue, add the element at the rear end of row K, where K is the row number as well as the priority number of that element. For example, if we have to insert an element R with priority number 3, then the priority queue will be given as shown in Fig.

FRONT	REAR	
3	3	1 [A]
1	3	2 [B C D]
4	1	3 [R E F]
4	1	4 [I G H]

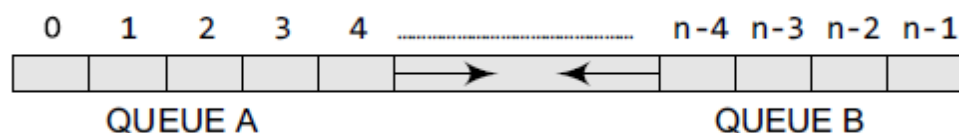
Deletion

To delete an element, we find the first nonempty queue and then process the front element of the first non-empty queue. In our priority queue, the first non-empty queue is the one with priority number 1 and the front element is A, so A will be deleted and processed first. In technical terms, find the element with the smallest K, such that $\text{FRONT}[K] \neq \text{NULL}$.

iii. Multiple Queues

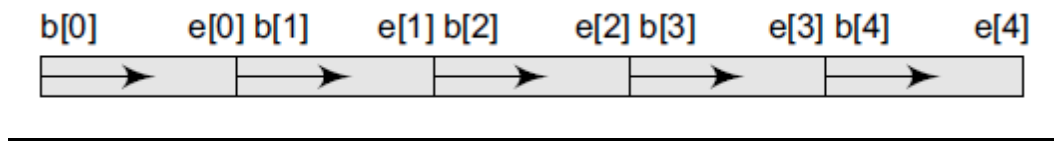
When we implement a queue using an array, the size of the array must be known in advance. If the queue is allocated less space, then frequent overflow conditions will be encountered. To deal with this problem, the code will have to be modified to reallocate more space for the array.

In case we allocate a large amount of space for the queue, it will result in sheer wastage of the memory. Thus, there lies a tradeoff between the frequency of overflows and the space allocated. So a better solution to deal with this problem is to have multiple queues or to have more than one queue in the same array of sufficient size. Figure below illustrates this concept.



In the figure, an array Queue[n] is used to represent two queues, Queue A and Queue B. The value of n is such that the combined size of both the queues will never exceed n. While operating on these queues, it is important to note one thing—queue A will grow from left to right, whereas queue B will grow from right to left at the same time.

Extending the concept to multiple queues, a queue can also be used to represent n number of queues in the same array. That is, if we have a QUEUE[n], then each queue I will be allocated an equal amount of space bounded by indices b[i] and e[i]. This is shown in Fig.



iv. DEQUES

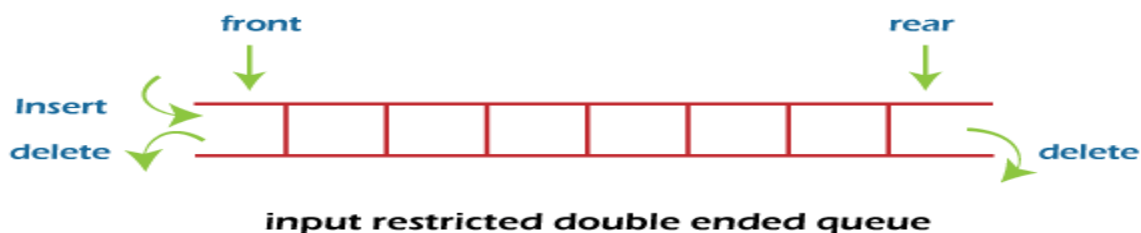
The deque stands for Double Ended Queue. Deque is a linear data structure where the insertion and deletion operations are performed from both ends. We can say that deque is a generalized version of the queue. It is also known as a *head-tail linked list* because elements can be added to or removed from either the front (head) or the back (tail) end. Thus, it does not follow FIFO rule (First In First Out).

However, no element can be added and deleted from the middle. In the computer's memory, a deque is implemented using either a circular array or a circular doubly linked list. In a deque, two pointers are maintained, LEFT and RIGHT, which point to either end of the deque. The elements in a deque extend from the LEFT end to the RIGHT end.

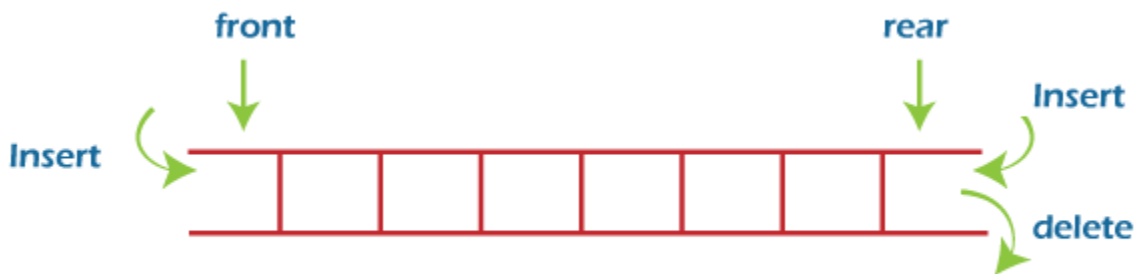


There are two variants of a double-ended queue. They include

- **Input restricted deque :-** In input restricted queue, insertion operation can be performed at only one end, while deletion can be performed from both ends.



- **Output restricted deque:-** In output restricted queue, deletion operation can be performed at only one end, while insertion can be performed from both ends.



Output restricted double ended queue

Operations performed on deque

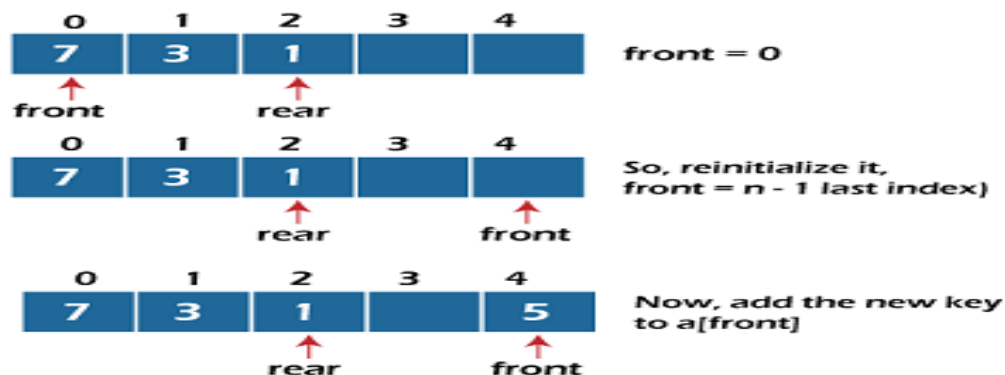
There are the following operations that can be applied on a deque -

- Insertion at front
- Insertion at rear
- Deletion at front
- Deletion at rear

Insertion at the front end

In this operation, the element is inserted from the front end of the queue. Before implementing the operation, we first have to check whether the queue is full or not. If the queue is not full, then the element can be inserted from the front end by using the below conditions -

- If the queue is empty, both rear and front are initialized with 0. Now, both will point to the first element.
- Otherwise, check the position of the front if the front is less than 1 ($\text{front} < 1$), then reinitialize it by **front = n - 1**, i.e., the last index of the array.



Algorithm insert at front

STEP 1: IF (FRONT=0 and REAR =MAX-1) or FRONT= REAR +1

Write OVERFLOW

goto step4

STEP 2: IF FRONT= -1 and REAR= -1

SET FRONT=REAR = 0

ELSE IF FRONT = 0

SET FRONT = MAX-1

ELSE

SET FRONT=FRONT-1

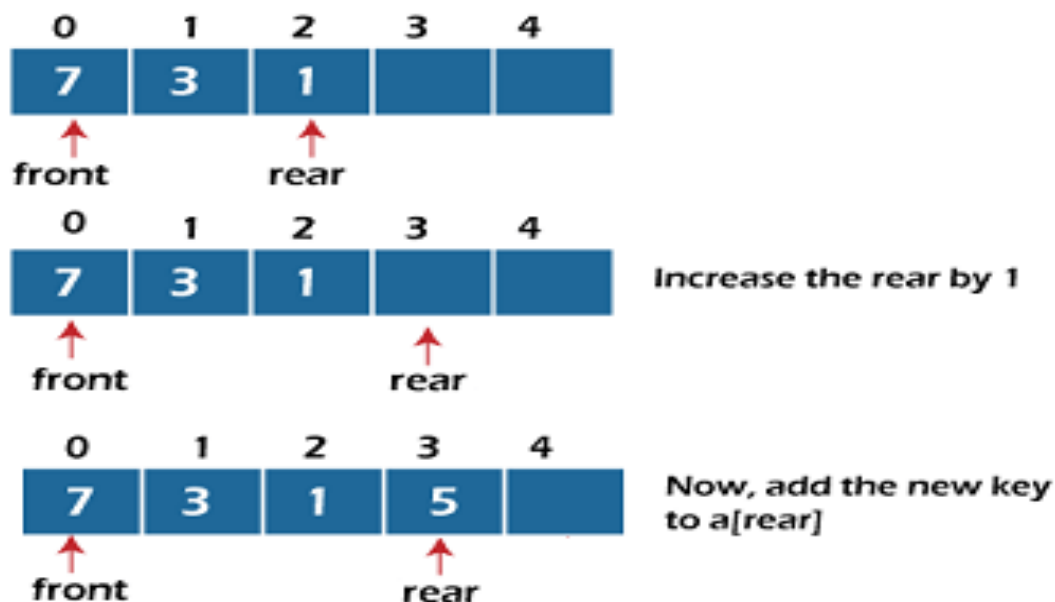
STEP 3: SET DEQUE[FRONT]=VAL

STEP 4 : EXIT

Insertion at the rear end

In this operation, the element is inserted from the rear end of the queue. Before implementing the operation, we first have to check again whether the queue is full or not. If the queue is not full, then the element can be inserted from the rear end by using the below conditions -

- If the queue is empty, both rear and front are initialized with 0. Now, both will point to the first element.
- Otherwise, increment the rear by 1. If the rear is at last index (or size - 1), then instead of increasing it by 1, we have to make it equal to 0.



Algorithm insert at rear

```
STEP 1: IF (FRONT=0 and REAR =MAX-1) or FRONT= REAR +1
        write OVER FLOW
        goto step 4
STEP 2: IF FRONT= -1 and REAR= -1
        SET FRONT=REAR = 0
        ELSE IF REAR =MAX-1 and FRONT!=0
        SET REAR =0
        ELSE
        SET REAR=REAR+1
STEP 3: SET DEQUE[REAR]=VAL
STEP 4: EXIT
```

Deletion at the front end

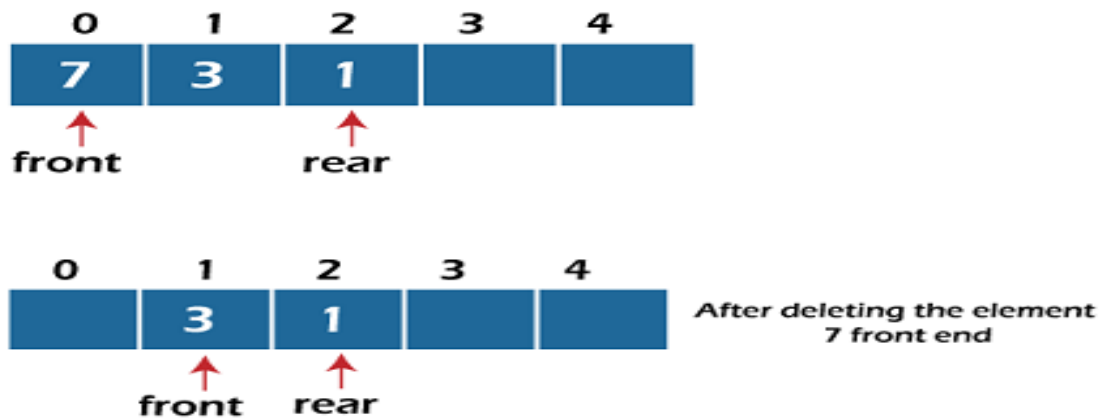
In this operation, the element is deleted from the front end of the queue. Before implementing the operation, we first have to check whether the queue is empty or not.

If the queue is empty, i.e., front = -1, it is the underflow condition, and we cannot perform the deletion. If the queue is not full, then the element can be inserted from the front end by using the below conditions -

If the deque has only one element, set rear = -1 and front = -1.

Else if front is at end (that means front = size - 1), set front = 0.

Else increment the front by 1, (i.e., front = front + 1).



Algorithm Deletion at the front

Step 1: IF FRONT = -1

Write " UNDERFLOW "

Goto Step 4

[END of IF]

Step 2: SET VAL = QUEUE[FRONT]

Step 3: IF FRONT = REAR

SET FRONT = REAR = -1

ELSE

IF FRONT = MAX -1

SET FRONT = 0

ELSE

SET FRONT = FRONT + 1

[END of IF]

[END OF IF]

Step 4: EXIT

Deletion at the rear end

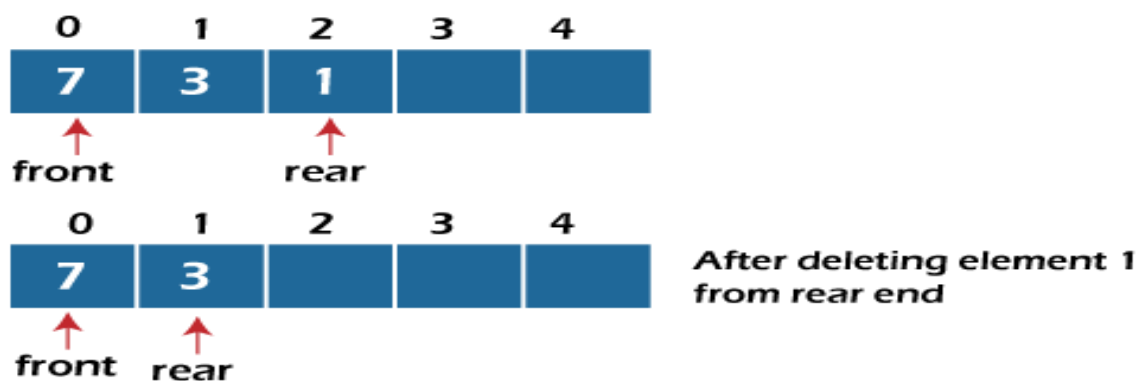
In this operation, the element is deleted from the rear end of the queue. Before implementing the operation, we first have to check whether the queue is empty or not.

If the queue is empty, i.e., front = -1, it is the underflow condition, and we cannot perform the deletion.

If the deque has only one element, set rear = -1 and front = -1.

If rear = 0 (rear is at front), then set rear = n - 1.

Else, decrement the rear by 1 (or, rear = rear -1).



Algorithm Deletion at the rear end

STEP 1: IF FRONT = -1 and REAR = -1

 write UNDER FLOW

 goto step

STEP 2: IF FRONT = REAR

 SET FRONT = REAR = -1

ELSE IF REAR = 0

 SET REAR = MAX-1

ELSE

 SET REAR = REAR-1

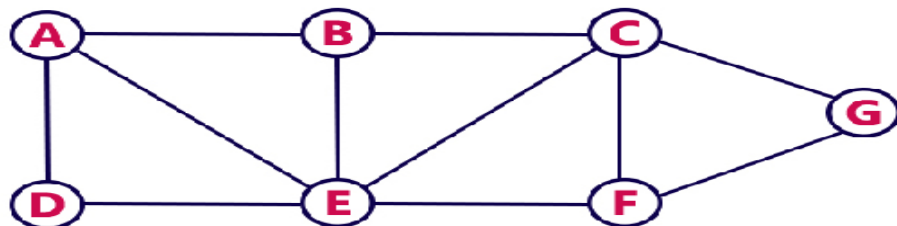
STEP 3: EXIT

APPLICATIONS OF QUEUE

Breadth-First Search (BFS)

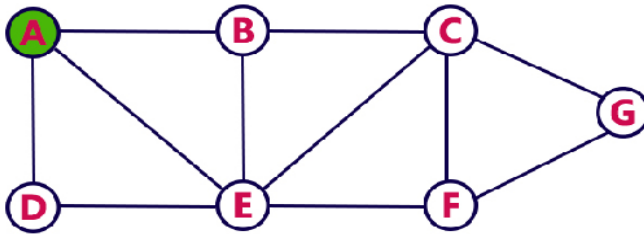
Breadth-first search is a graph traversal algorithm that starts traversing the graph from the root node and explores all the neighboring nodes. Then, it selects the nearest node and explores all the unexplored nodes. While using BFS for traversal, any node in the graph can be considered as the root node.

Consider the following example graph to perform BFS traversal



Step 1:

- Select the vertex **A** as starting point (visit **A**).
- Insert **A** into the Queue.

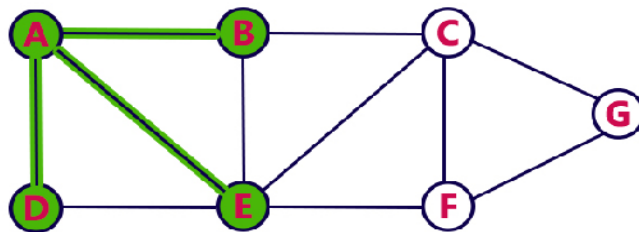


Queue



Step 2:

- Visit all adjacent vertices of **A** which are not visited (**D, E, B**).
- Insert newly visited vertices into the Queue and delete A from the Queue..

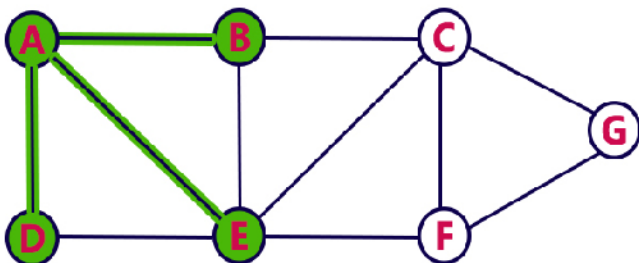


Queue



Step 3:

- Visit all adjacent vertices of **D** which are not visited (there is no vertex).
- Delete D from the Queue.

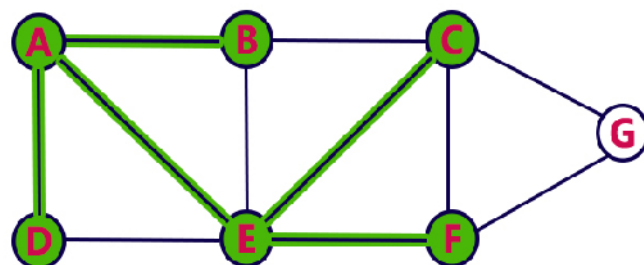


Queue



Step 4:

- Visit all adjacent vertices of **E** which are not visited (**C, F**).
- Insert newly visited vertices into the Queue and delete E from the Queue.

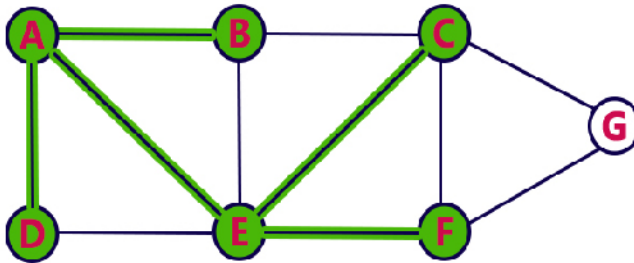


Queue



Step 5:

- Visit all adjacent vertices of **B** which are not visited (**there is no vertex**).
- Delete **B** from the Queue.

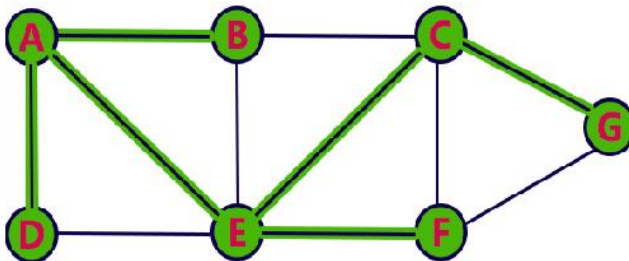


Queue



Step 6:

- Visit all adjacent vertices of **C** which are not visited (**G**).
- Insert newly visited vertex into the Queue and delete **C** from the Queue.

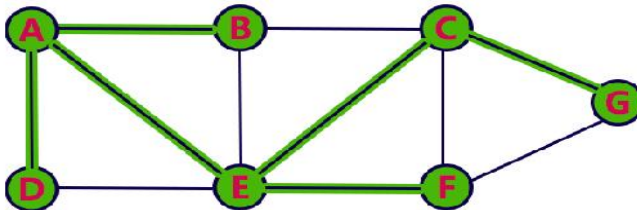


Queue



Step 7:

- Visit all adjacent vertices of **F** which are not visited (**there is no vertex**).
- Delete **F** from the Queue.

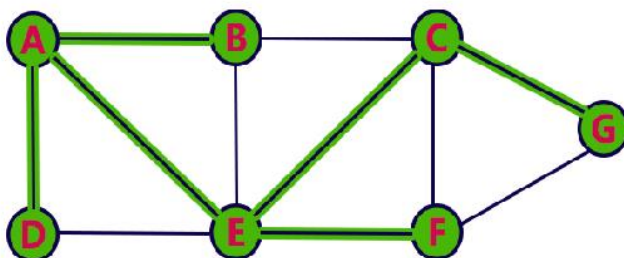


Queue



Step 8:

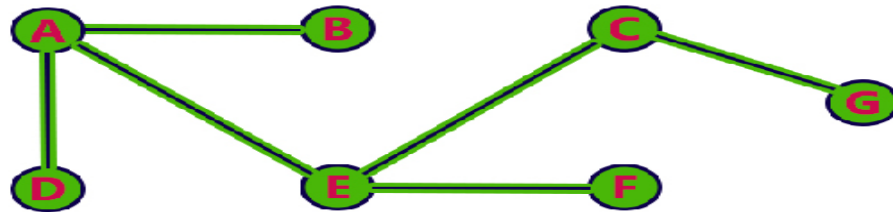
- Visit all adjacent vertices of **G** which are not visited (**there is no vertex**).
- Delete **G** from the Queue.



Queue



- Queue became Empty. So, stop the BFS process.
- Final result of BFS is a Spanning Tree as shown below...



Algorithm for breadth-first search

- Step 1:** SET STATUS = 1 (ready state) for each node in G
- Step 2:** Enqueue the starting node A and set its STATUS = 2 (waiting state)
- Step 3:** Repeat Steps 4 and 5 until QUEUE is empty
- Step 4:** Dequeue a node N. Process it and set its STATUS = 3 (processed state).
- Step 5:** Enqueue all the neighbours of N that are in the ready state (whose STATUS = 1) and set their STATUS = 2 (waiting state)
- [END OF LOOP]
- Step 6:** EXIT

Scheduling

Queues have wide ranging applications in today's world. The major use of queues lies in the process scheduling of an operating system. Process scheduling can be performed in different ways and various algorithms are adopted for scheduling depending on the no. of processes waiting to be executed and execution time of every process. Common scheduling algorithms include First Come First Served (FCFS) algorithm, Round Robin Scheduling algorithm and Priority based Scheduling algorithm etc.

Round Robin Scheduling

The Round Robin algorithm executes jobs for specific quantities of time called time quantum. Thus, identical time quantum slots will be allotted to all arriving processes. Processes having their burst times greater than the time quantum will be interrupted and moved towards the end of the ready queue while the following process in the queue gets executed. A large value of the time quantum will make it behave as a FCFS algorithm whereas a small value for the time quantum will result in increased no. of context switches.

The algorithm uses enqueue operation of queues to store the information of processes and it uses dequeue operation to remove a process from the ready queue when its execution is complete.

Example :-

SNO	PROCESS ID	ARRIVAL TIME	BURST TIME
1	P1	0	7
2	P2	1	4
3	P3	2	15
4	P4	3	11
5	P5	4	20
6	P6	4	9

Assume Time Quantum TQ = 5

Ready Queue:

P1, P2, P3, P4, P5, P6, P1, P3, P4, P5, P6, P3, P4, P5

Gantt chart:

P1	P2	P3	P4	P5	P6	P1	P3	P4	P5	P6	P3	P4	P5
0	5	9	14	19	24	29	31	36	41	46	50	55	66

Process Id	Arrival Time(AT)	Burst Time (BT)	Completion Time (CT)	Turn Arround Time TRT=CT-AT	Waiting Time WT=(TRT-BT)
P1	0	7	31	31	24
P2	1	4	9	8	4
P3	2	15	55	53	38
P4	3	11	56	53	42
P5	4	20	66	62	42
P6	4	9	50	46	37

Algorithm

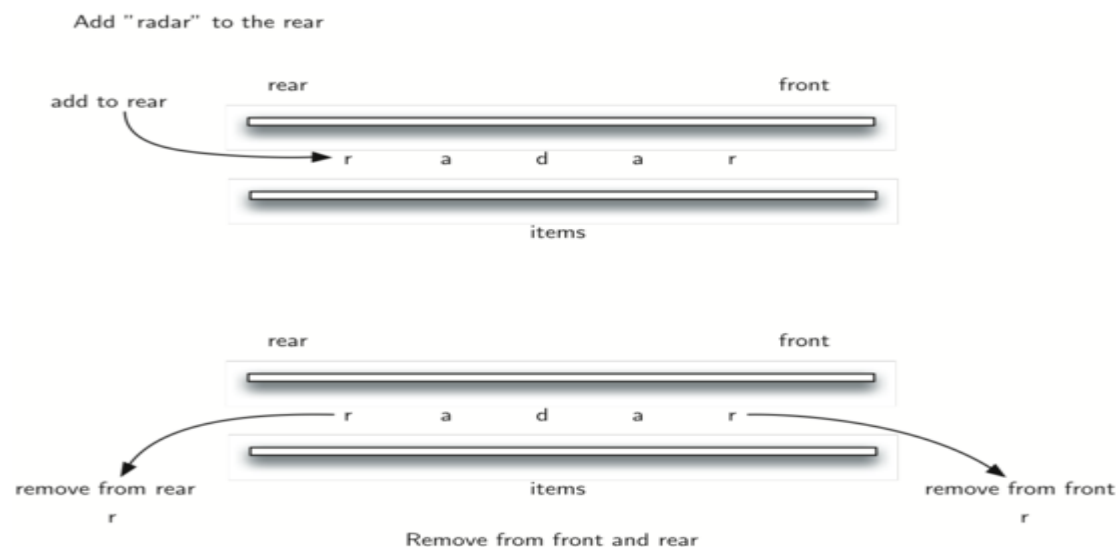
- Step 1:** Enter the no. of processes (NOP) along with their burst times (bt).
Step 2: Store the burst times in a ready queue 'q' by using the enqueue function, enqueue(q, bt, index).
Step 3: Enter the value of time quantum quant.
Step 4: if (NOP>0), go to step 5.
 else go to step 7.
Step 5: Check if remaining burst time (RBT) <= quant. if yes, the process gets executed completely. Otherwise, it is executed for a time period quant.
Step 6: if (RBT==0), display the details of that process, display (q, index) and remove the process from the queue, dequeue (q, index).
Step 7: Calculate AWT and ATAT of all the processes.

APPLICATIONS OF DEQUES

Palindrome-Checker

An interesting problem that can be easily solved using the deque data structure is the classic palindrome problem. A palindrome is a string that reads the same forward and backward, for example, radar, toot, and madam. We would like to construct an algorithm to input a string of characters and check whether it is a palindrome.

The solution to this problem will use a deque to store the characters of the string. We will process the string from left to right and add each character to the rear of the deque. At this point, the deque will be acting very much like an ordinary queue. However, we can now make use of the dual functionality of the deque. The front of the deque will hold the first character of the string and the rear of the deque will hold the last character



Since we can remove both of them directly, we can compare them and continue only if they match. If we can keep matching first and the last items, we will eventually either run out of characters or be left with a deque of size 1 depending on whether the length of the original string was even or odd. In either case, the string must be a palindrome. The complete function for palindrome-checking appears in ActiveCode 1.

Algorithm_Palindrome_check

BEGIN

If front = rear

Write “no operation performed”

else

while size(character_deque) > 1:

first = character_deque.dequeueLeft

last = character_deque.dequeueRight

if first != last:

return False

end if

end of while

end if

return True

END

Implementation of Priority Queue using array of structures and circular queue concept

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define MAX_SIZE 5
```

```
// Structure definition for priority queue elements
```

```
struct PriorityQ {
```

```
    int item;
```

```
    int priority;
```

```
};
```

```

// Array of structures to represent the priority queue
struct PriorityQ PQ[MAX_SIZE];

// Global variables to represent front and rear of the queue
int front = -1, rear = -1;

// Function prototypes
void enqueue(int, int);
void dequeue();
void display();

int main() {
    int choice, item, prio;

    while (1) {
        printf("\n-----\n");
        printf("Priority Queue Menu\n");
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Display Queue\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter item and priority: ");
                scanf("%d %d", &item, &prio);
                enqueue(item, prio);
                break;
            case 2:
                dequeue();
                break;
            case 3:
                display();
                break;
            case 4:
                printf("Exiting...\n");
                exit(0);
            default:
                printf("Invalid choice\n");
        }
    }
    return 0;
}

```

```

// Function to enqueue an element into the priority queue
void enqueue(int x, int p) {
    if ((rear + 1) % MAX_SIZE == front) {
        printf("Overflow...Priority Queue is Full of elements.\n");
        return;
    }

    if (front == -1 && rear == -1) {
        front = rear = 0;
        PQ[rear].item = x;
        PQ[rear].priority = p;
    } else {
        rear = (rear + 1) % MAX_SIZE;
        PQ[rear].item = x;
        PQ[rear].priority = p;
    }

    // Rearrange elements based on priority
    int i = rear;
    while (i != front && PQ[(i - 1 + MAX_SIZE) % MAX_SIZE].priority > PQ[i].priority) {
        // Swap items and priorities
        struct PriorityQ temp = PQ[i];
        PQ[i] = PQ[(i - 1 + MAX_SIZE) % MAX_SIZE];
        PQ[(i - 1 + MAX_SIZE) % MAX_SIZE] = temp;

        i = (i - 1 + MAX_SIZE) % MAX_SIZE;
    }
}

// Function to dequeue an element from the priority queue
void dequeue() {
    if (front == -1 && rear == -1) {
        printf("Underflow...Priority Queue is empty.\n");
        return;
    }

    printf("%d dequeued\n", PQ[front].item);

    if (front == rear) {
        front = rear = -1;
    } else {
        front = (front + 1) % MAX_SIZE;
    }
}

```

```
// Function to display the elements of the priority queue
void display() {
    if (front == -1 && rear == -1) {
        printf("Priority queue is empty\n");
        return;
    }

    printf("ITEMS:\t\tPRIORITIES:\n");
    int i = front;
    do {
        printf("%5d\t\t%5d\n", PQ[i].item, PQ[i].priority);
        i = (i + 1) % MAX_SIZE;
    } while (i != (rear + 1) % MAX_SIZE);
}
```

Output:-

Priority Queue Menu

1. Enqueue
2. Dequeue
3. Display Queue
4. Exit

Enter your choice: 1

Enter item and priority: 11

1

Priority Queue Menu

1. Enqueue
2. Dequeue
3. Display Queue
4. Exit

Enter your choice: 1

Enter item and priority: 12

1

Priority Queue Menu

1. Enqueue
2. Dequeue
3. Display Queue
4. Exit

Enter your choice: 1

Enter item and priority: 22

2

Priority Queue Menu

1. Enqueue
2. Dequeue
3. Display Queue
4. Exit

Enter your choice: 1

Enter item and priority: 33

3

Priority Queue Menu

1. Enqueue
2. Dequeue
3. Display Queue
4. Exit

Enter your choice: 3

ITEMS: PRIORITIES:

11 1

12 1

22 2

33 3

Priority Queue Menu

1. Enqueue
2. Dequeue
3. Display Queue
4. Exit

Enter your choice: 2

11 dequeued

Priority Queue Menu

1. Enqueue
2. Dequeue
3. Display Queue
4. Exit

Enter your choice: 3

ITEMS: PRIORITIES:

12 1

22 2

33 3